Informe del laboratorio SOLID Race Lab

Johan Alexander Peña Ortiz

Ingeniería de Software II · Universidad Nacional de Colombia · 2026-2

Link repositorio: https://github.com/jopenao-14005/solid-race-lab

Introducción

Este informe resume lo realizado en cada uno de los cinco ejercicios del taller SOLID Race Lab. En el taller se completa el código de una carrera de vehículos animada en consola, de modo que cada ejercicio aplica un principio SOLID distinto. Al final, las 21 pruebas automáticas del proyecto pasan correctamente.

Ejercicio 1 – SRP (Responsabilidad Única)

Problema: La clase Car hacía tres tareas a la vez: guardar su estado, imprimir su estado y escribir un registro en disco.

Qué se hizo:

- Car quedó solo con su estado (nombre, velocidad, símbolo, posición) y con el método move(), que únicamente avanza la posición.
- Se creó la clase RaceLogger, encargada de registrar y guardar resultados: record(tick, racers) guarda las posiciones de cada tick, save(path) las escribe en un archivo y la propiedad entries devuelve las entradas como lista.
- En main(), logger.record se pasa como callback on_tick a Track.run(), y logger.save() se llama al terminar la carrera.

Resultado: cada clase tiene un único motivo de cambio; si mañana se quiere registrar en JSON, solo se modifica RaceLogger.

Ejercicio 2 – OCP (Abierto/Cerrado)

Problema: Había que modificar algunos vehículos sin modificar Vehicle, Car, Truck ni Track.

Qué se hizo:

- Motorcycle avanza en cada tick una cantidad aleatoria entre 1 y 9, lo que la hace rápida pero errática.
- Bicycle lleva un contador de ticks y avanza max(1, 6 − ticks // 2): empieza fuerte y se va cansando, sin bajar nunca de 1.
- Ambas son subclases ya existentes de Vehicle; no se tocó ninguna otra clase.

Resultado: la carrera muestra cuatro patrones de movimiento distintos, logrados sólo por extensión mediante polimorfismo.

Ejercicio 3 – LSP (Sustitución de Liskov)

Problema: UnreliableCar rompía el contrato de Vehicle: podía lanzar excepciones o retroceder.

Qué se hizo:

- Se rediseñó move() para que la falta de fiabilidad sea compatible con el contrato: con 30 % de probabilidad el auto simplemente no avanza; en los demás casos avanza 5.
- La posición nunca disminuye y nunca se lanza una excepción.
- Vehicle y Track quedaron sin cambios.

Resultado: UnreliableCar puede sustituir a cualquier Vehicle sin romper la carrera, y Track no necesita try/except ni correcciones de posición.

Ejercicio 4 – ISP (Segregación de Interfaces)

Problema: Una interfaz «gorda» (VehicleActions) obligaba a los vehículos a implementar métodos que no les corresponden, como fly() en un auto o una bicicleta.

Qué se hizo:

- Se reemplazó por cuatro interfaces pequeñas, cada una con un único método abstracto: Movable (move), Refuelable (refuel), Flyable (fly) y Pedalable (pedal_harder).
- GasCar implementa solo Movable y Refuelable; Bicycle solo Movable y Pedalable. Se eliminaron los métodos que lanzaban «no puedo hacer eso».
- Se agregó la clase Drone (Movable y Flyable) y se incluyó en la carrera de main().

Resultado: cada clase depende solo de lo que realmente usa, y agregar un vehículo nuevo no obliga a modificar los existentes.

Ejercicio 5 – DIP (Inversión de Dependencias)

Problema: La clase Race construía por sí misma sus vehículos concretos, por lo que cambiar los participantes exigía editar Race.

Qué se hizo:

- Race ahora recibe los corredores por el constructor (self.racers) y, opcionalmente, la pista (track); ya no conoce ninguna clase concreta de vehículo.
- Se agregó la clase RocketSled, que cumple el contrato de corredor (name, symbol, position, move()).
- En main() se corren dos carreras con plantillas distintas (SportsCar y DeliveryVan; RocketSled y DeliveryVan) usando la misma clase Race.

Resultado: Race depende de la abstracción Racer y no de implementaciones concretas, por lo que también es más fácil de probar.

Conclusión

Los cinco ejercicios muestran cómo SOLID produce un código más ordenado y flexible: responsabilidades separadas (SRP), extensión sin modificación (OCP), subclases intercambiables (LSP), interfaces pequeñas (ISP) y dependencias de abstracciones (DIP). La validación con pytest (21 pruebas aprobadas) y la ejecución de las demos confirman que todos los ejercicios quedaron resueltos. A continuación la prueba de ello:

```
(.venv) PS C:\Users\johan\Downloads\solid-race-lab-20261001T022116Z-1-001\solid-race-lab> pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\johan\Downloads\solid-race-lab-20261001T022116
Z-1-001\solid-race-lab\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\johan\Downloads\solid-race-lab-20261001T022116Z-1-001\solid-race-lab
configfile: pytest.ini
testpaths: tests
collected 21 items

tests/test_ex1_srp.py::test_car_move_does_not_print PASSED                                          [  4%]
tests/test_ex1_srp.py::test_car_move_only_updates_position PASSED                                   [  9%]
tests/test_ex1_srp.py::test_car_move_does_not_touch_disk PASSED                                     [ 14%]
tests/test_ex1_srp.py::test_race_logger_exists PASSED                                               [ 19%]
tests/test_ex1_srp.py::test_race_logger_records_and_saves PASSED                                    [ 23%]
tests/test_ex1_srp.py::test_main_runs_end_to_end PASSED                                             [ 28%]
tests/test_ex2_ocp.py::test_car_and_truck_unchanged PASSED                                          [ 33%]
tests/test_ex2_ocp.py::test_motorcycle_and_bicycle_are_vehicles PASSED                              [ 38%]
tests/test_ex2_ocp.py::test_motorcycle_moves_variably PASSED                                        [ 42%]
tests/test_ex2_ocp.py::test_bicycle_gets_slower_over_time PASSED                                    [ 47%]
tests/test_ex3_lsp.py::test_unreliable_car_is_a_vehicle PASSED                                      [ 52%]
tests/test_ex3_lsp.py::test_unreliable_car_never_moves_backwards_or_raises PASSED                   [ 57%]
tests/test_ex3_lsp.py::test_track_can_run_a_full_race_without_crashing PASSED                       [ 61%]
tests/test_ex4_isp.py::test_small_focused_interfaces_exist PASSED                                   [ 66%]
tests/test_ex4_isp.py::test_gas_car_only_implements_relevant_interfaces PASSED                      [ 71%]
tests/test_ex4_isp.py::test_bicycle_only_implements_relevant_interfaces PASSED                      [ 76%]
tests/test_ex4_isp.py::test_drone_exists_and_only_implements_relevant_interfaces PASSED             [ 80%]
tests/test_ex5_dip.py::test_race_constructor_accepts_racers_from_outside PASSED                     [ 85%]
tests/test_ex5_dip.py::test_race_does_not_hardcode_its_roster PASSED                                [ 90%]
tests/test_ex5_dip.py::test_race_runs_with_an_arbitrary_roster PASSED                               [ 95%]
tests/test_ex5_dip.py::test_race_works_with_two_different_rosters_unmodified PASSED                 [100%]

============================================== 21 passed in 0.11s ==============================================
(.venv) PS C:\Users\johan\Downloads\solid-race-lab-20261001T022116Z-1-001\solid-race-lab>
```